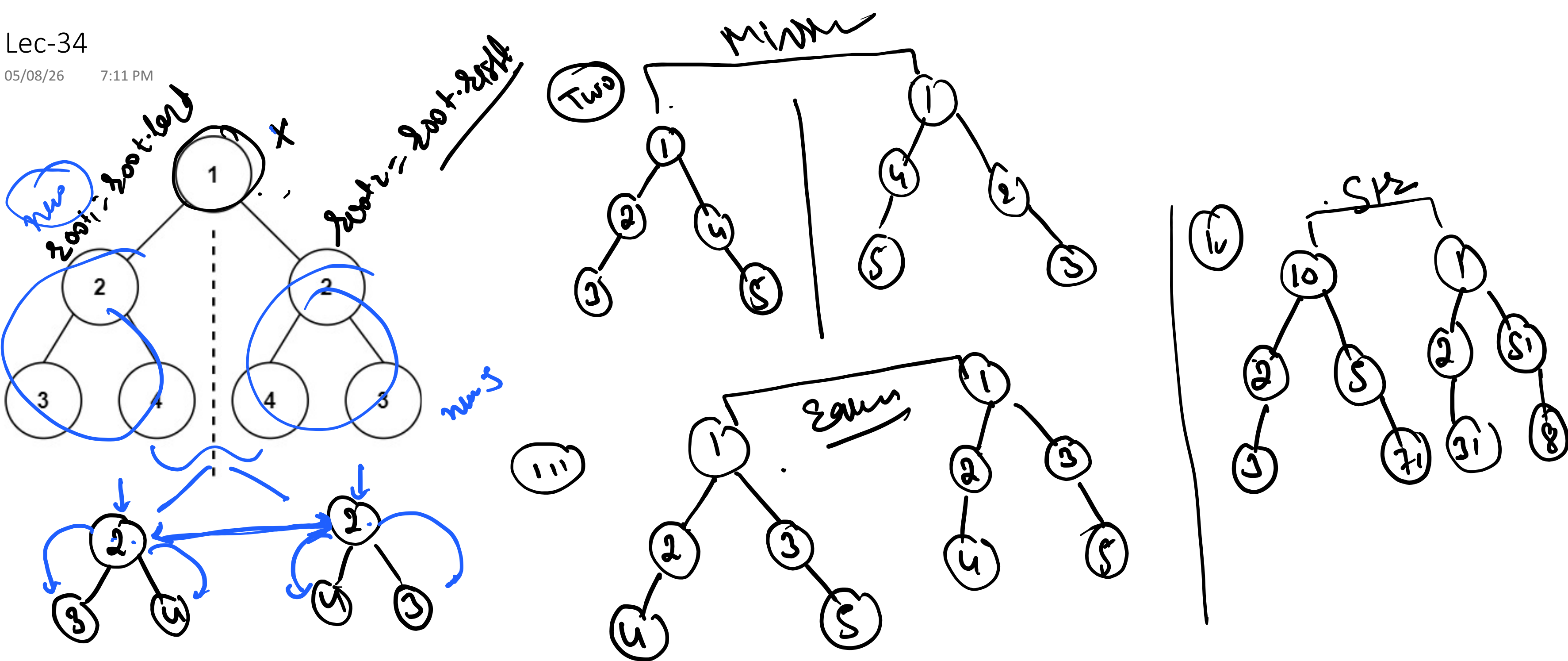
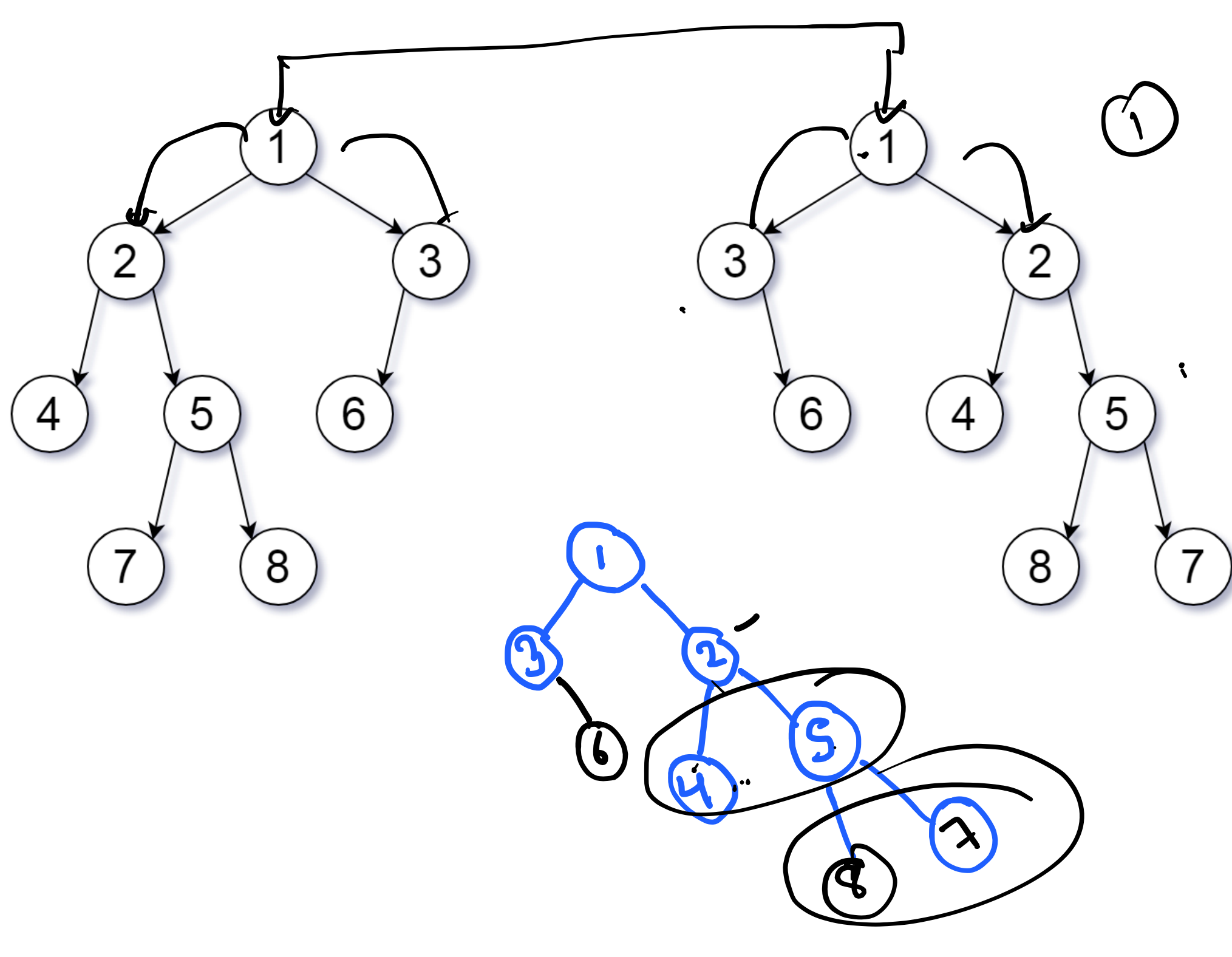
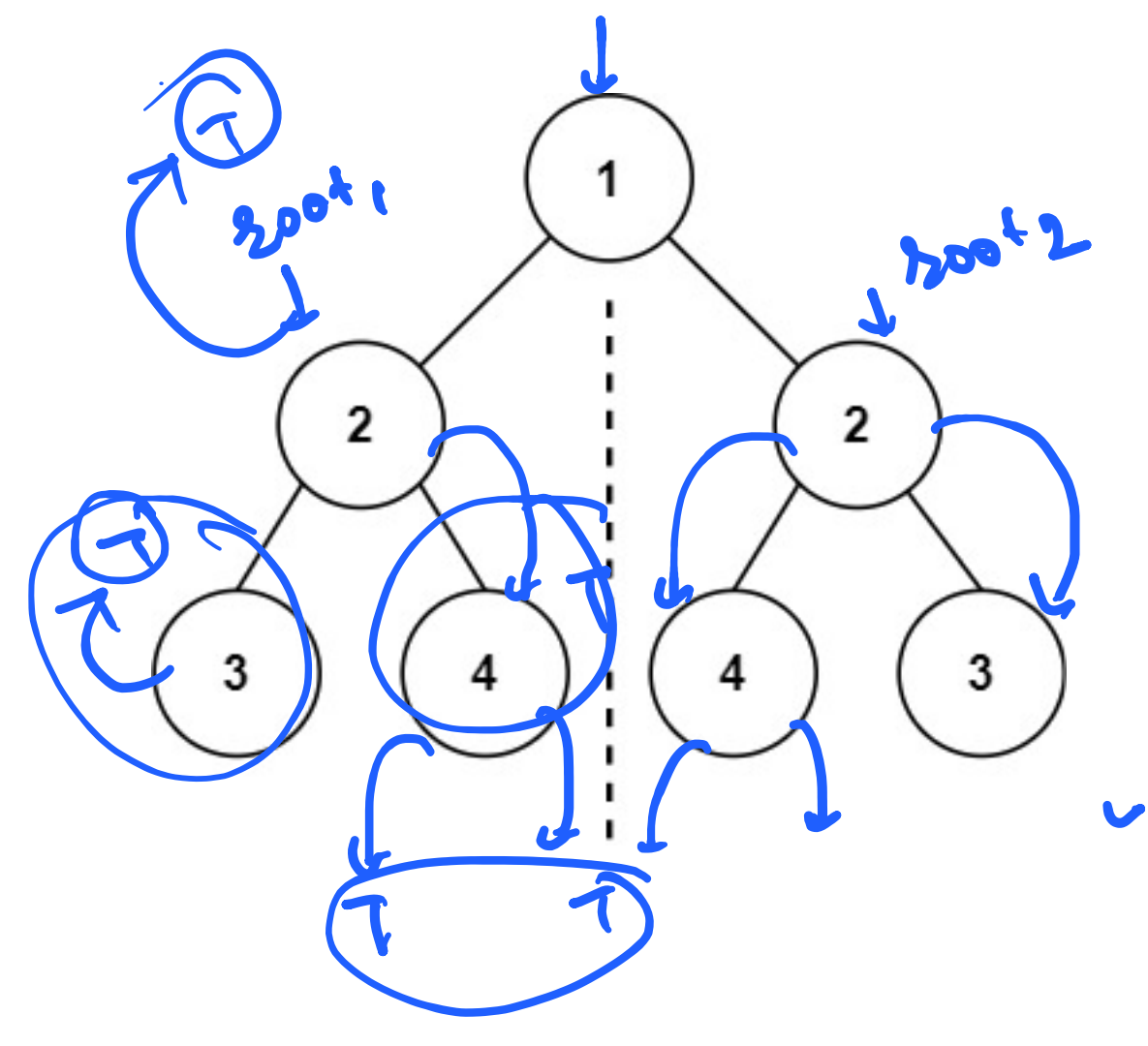


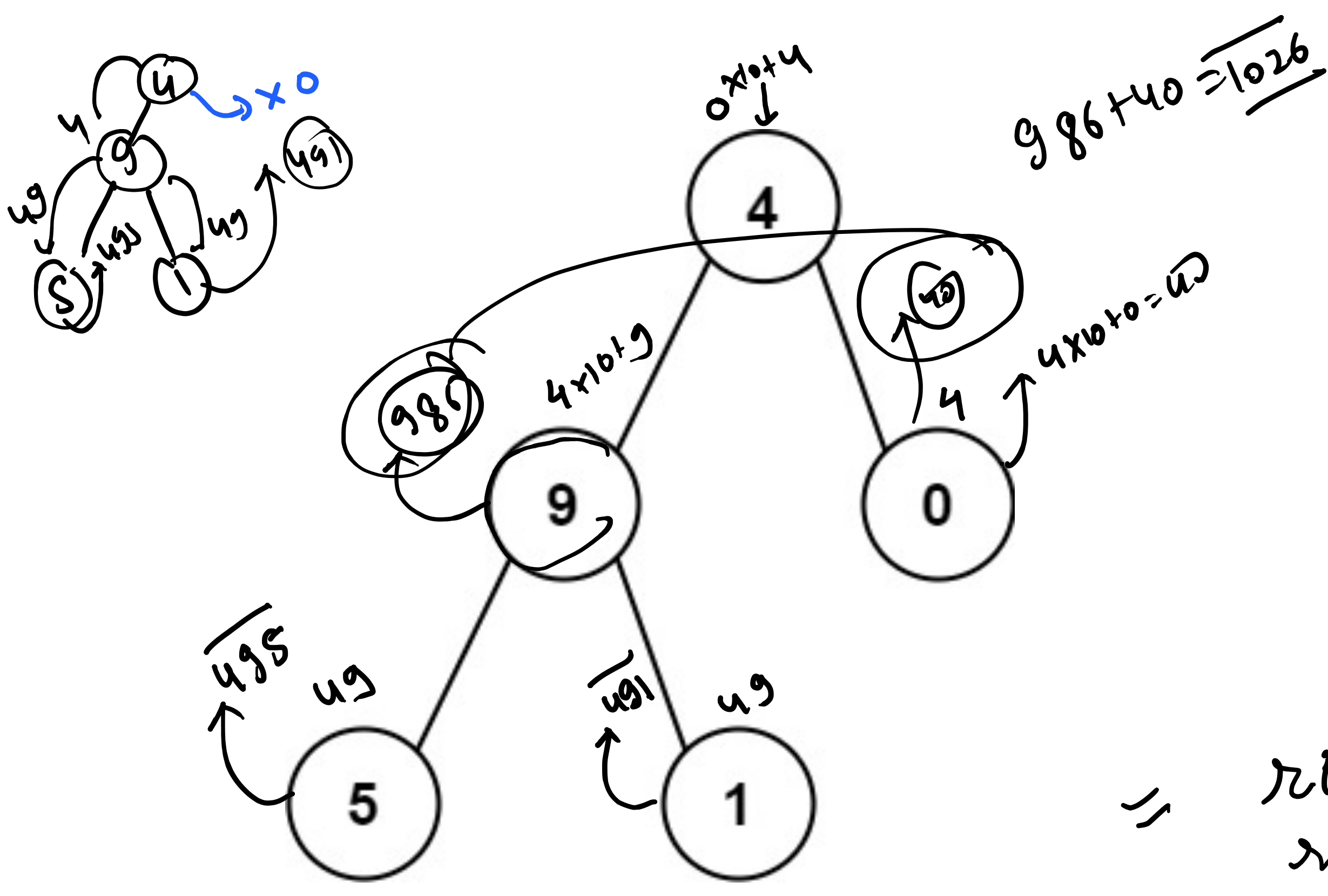
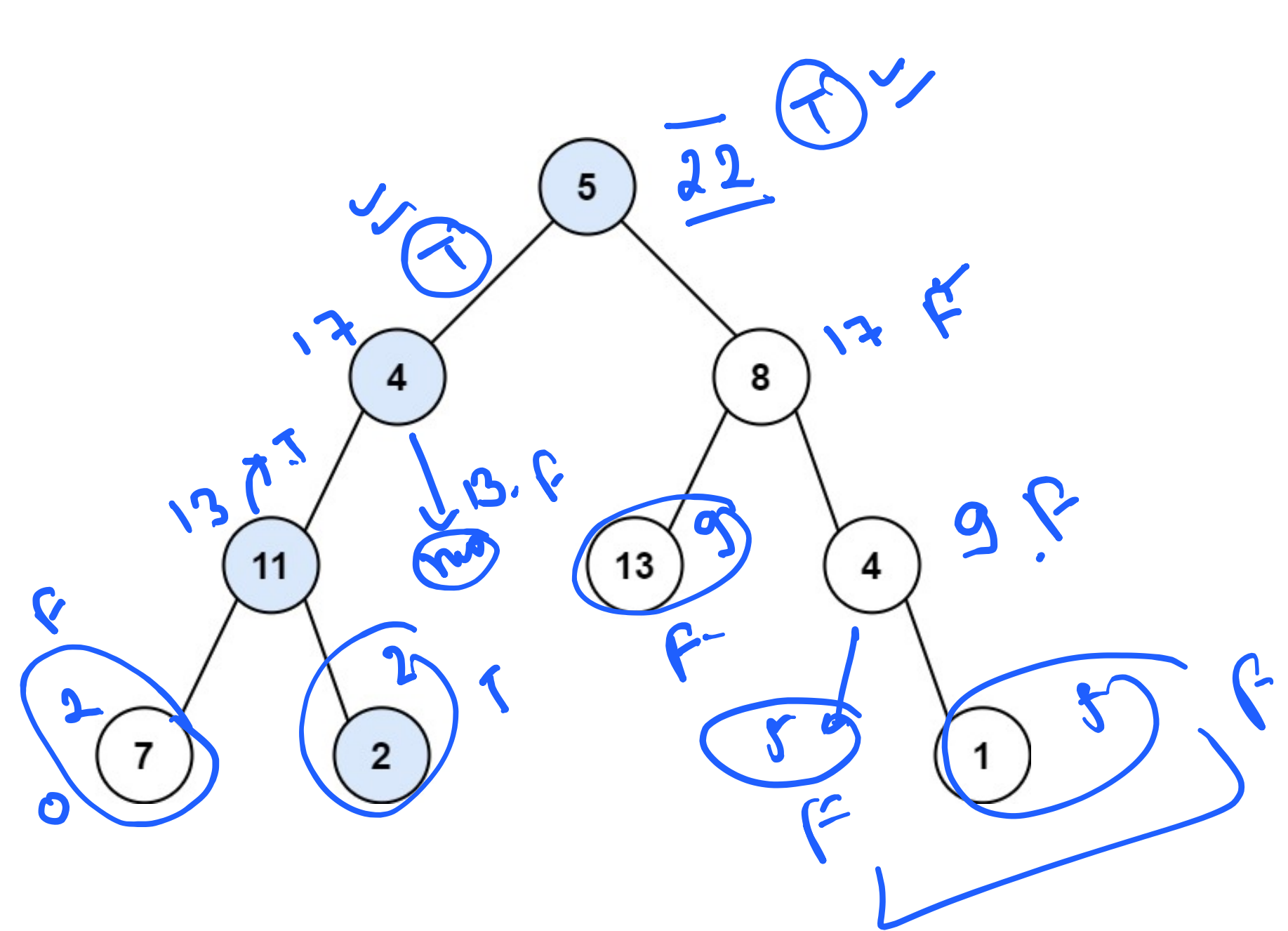
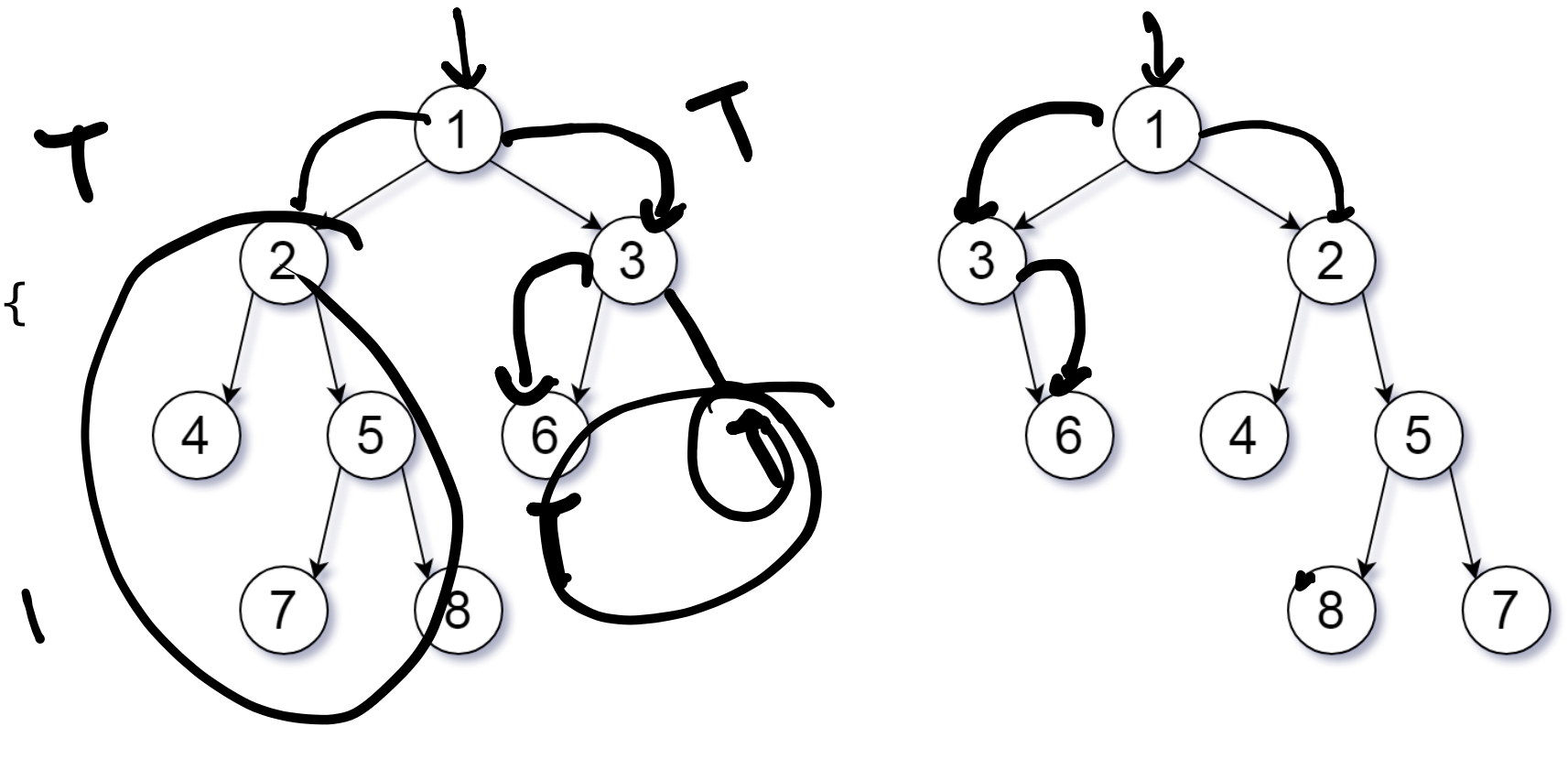


```
public boolean isSymmetric(TreeNode root) {
    return SymmetricTree(root.left, root.right);
}

public boolean SymmetricTree(TreeNode root1, TreeNode root2) {
    if (root1 == null && root2 == null) {
        return true;
    }
    if (root1 == null || root2 == null) {
        return false;
    }
    if (root1.val != root2.val) {
        return false;
    }
    boolean left = SymmetricTree(root1.left, root2.right);
    boolean right = SymmetricTree(root1.right, root2.left);
    return left && right;
}
```



```
public boolean flipEquiv(TreeNode root1, TreeNode root2) {
    if (root1 == null && root2 == null) {
        return true;
    }
    if (root1 == null || root2 == null) {
        return false;
    }
    if (root1.val != root2.val) {
        return false;
    }
    boolean flip = flipEquiv(root1.left, root2.right) && flipEquiv(root1.right, root2.left);
    boolean noflip = flipEquiv(root1.left, root2.left) && flipEquiv(root1.right, root2.right);
    return flip || noflip;
}
```



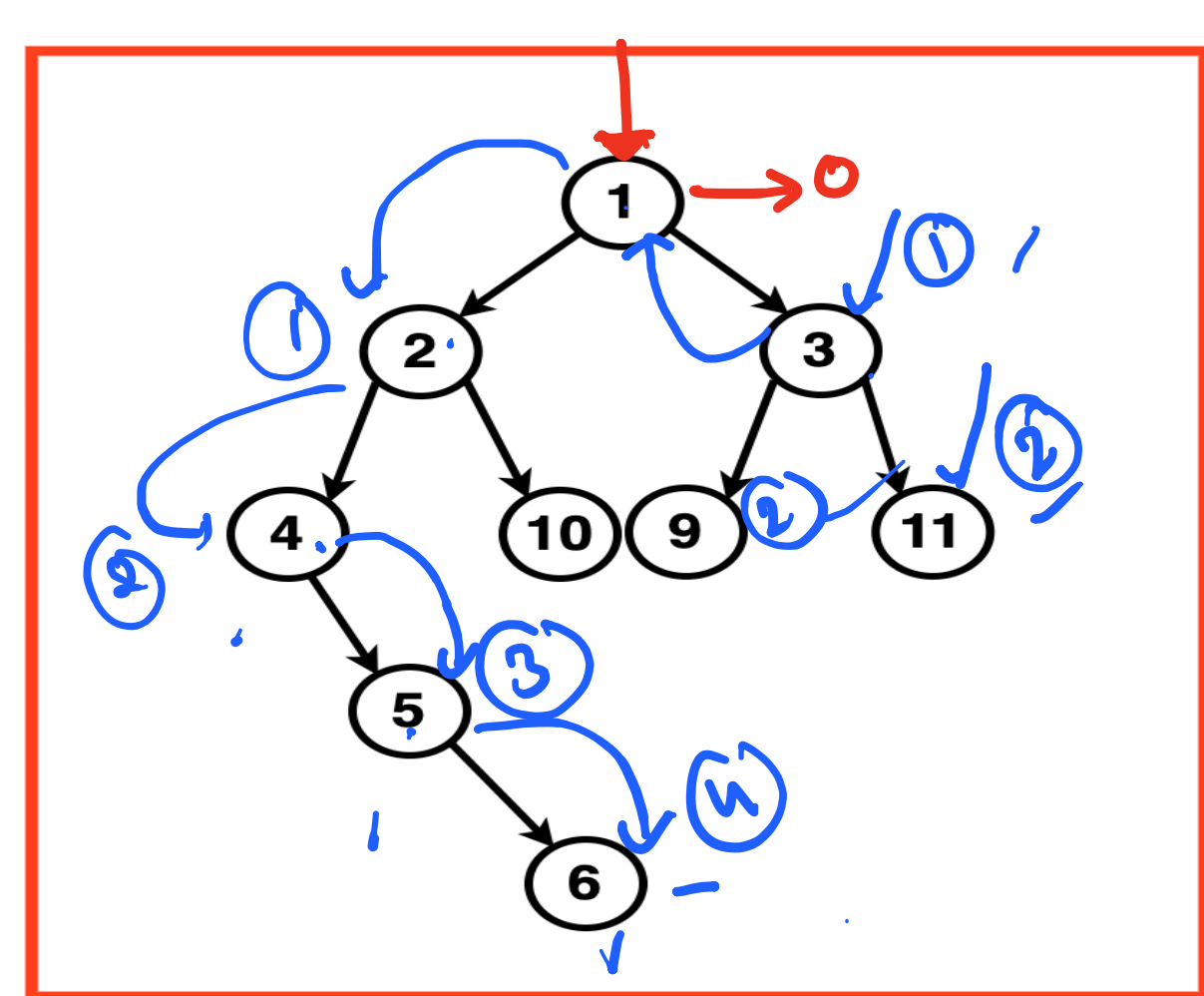
= root left + sum root right

depth sum

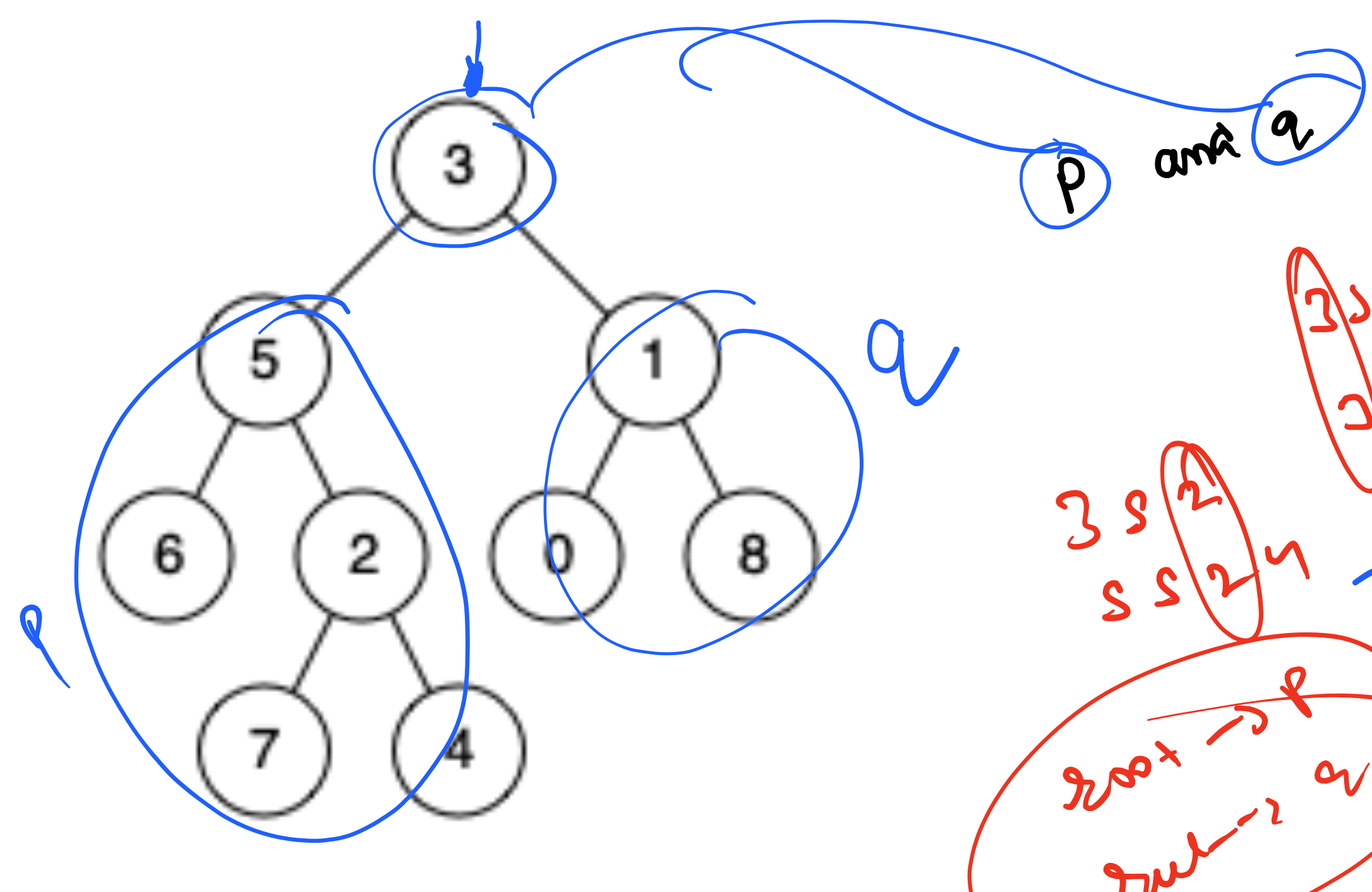
max depth = 0 x 2 x 4

2 x 6 x 6

C, 1, 3, 11, 18, 6



T = O(n)



3 x 6 x 6

code logic

p = 2 q = 4

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null) {
        return null;
    }
    if (root == p || root == q) {
        return root;
    }
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left != null && right != null) {
        return root;
    }
    else if (left == null) {
        return right;
    }
    else {
        return left;
    }
}
```

